

Automating Proof Search when Equality is a Logical Connective

Kaustuv Chaudhuri, Arunava Gantait, Dale Miller
IJCAR, 27 July 2026

The Setting: Equality as a logical connective

The Focus: Proof Search

Our Work: Automating Proof Search

Conclusion

The Setting: Equality as a logical connective

Adding equality as a logical connective

We use a notion of equality due independently to Schroeder-Heister and Girard¹

We add the following rules:

$$\frac{}{\Sigma :: \Gamma \vdash t = t} =R \qquad \frac{\{\Sigma \sigma :: \Gamma \sigma \vdash A \sigma \mid \sigma \in csu(s, t)\}}{\Sigma :: s = t, \Gamma \vdash A} =L$$

Here **csu** = *complete set of unifiers*

Note that **unification** is now a **logical feature** and not just an implementation detail.

¹Also called **definitional equality**

The Proof System $LJ_{\omega}^=$

Definition of $LJ_{\omega}^=$

$$LJ_{\omega}^= = LJ + \text{equality rules} = LJ + =L + =R$$

The ω stands for *arbitrary orders of quantifiers*

Why definitional equality?

Abella's underlying logic $\mathcal{G}$ uses definitional equality

This proof system can encode (intuitionistic) Robinson Arithmetic $\mathcal{Q}$

Cut is eliminable (unlike axiomatic equality)²

²See McDowell and Miller, *Cut-elimination for a logic with definitions and induction*, Theoretical Computer Science, 2000

The Focus: Proof Search

Introducing SLIM

We **restrict the shape of sequents** to a suitably expressive sublogic.

This gives us the logical framework **SLIM**:

Specification Logic for an Intuitionistic **M**etalogic

Introducing SLIM

We **restrict the shape of sequents** to a suitably expressive sublogic.

This gives us the logical framework **SLIM**:

SLIM Grammar

Goal formulas G and **Definite clauses** D :

$$G := \mathbf{t} \mid \mathbf{f} \mid A \mid G_1 \wedge G_2 \mid \exists_{\tau} x. G \mid \forall_{\sigma} x. G \mid t = s \mid t = s \supset G$$

$$D := G \supset A \mid \forall_{\tau} x. D$$

where σ is necessarily of order 0 but τ is of order 0 or 1³

³to allow first-order quantification in object logics

Why SLIM?

Main goal: Use SLIM as a **logical framework**, i.e., specify first-order logical systems in it

Why SLIM?

Main goal: Use SLIM as a **logical framework**, i.e., specify first-order logical systems in it

SLIM = minimal subset of $LJ_{\omega}^=$ that allows this

Why SLIM?

Main goal: Use SLIM as a **logical framework**, i.e., specify first-order logical systems in it

SLIM = minimal subset of $LJ_{\omega}^=$ that allows this

Restriction on types helps with proof search by limiting the size of csu's

Using SLIM

As stated before, SLIM is useful for specifying logical systems, especially those with equality

Use meta-level equality to encode object-level equality

Consider specifying the rules for a one-sided sequent calculus for a logic with equality. The following *D*-formulas concern just equality.

```
% equality
```

```
seq (eq T T :: Gam) :- seq Gam.
```

```
seq (neq S T :: Gam) :- (S = T) => seq Gam.
```

The Proof Search Problem

The Proof Search Problem in SLIM

Provide a sound and complete proof search strategy for SLIM sequents of the form $\Sigma :: \mathcal{P} \vdash G$ where $\mathcal{P}$ is a **logic program** - a set of $\mathcal{D}$ -formulas

The Proof Search Problem

The Proof Search Problem in SLIM

Provide a sound and complete proof search strategy for SLIM sequents of the form $\Sigma :: \mathcal{P} \vdash G$ where $\mathcal{P}$ is a **logic program** - a set of $\mathcal{D}$ -formulas

The proof search problem for SLIM is undecidable

The Proof Search Problem

The Proof Search Problem in SLIM

Provide a sound and complete proof search strategy for SLIM sequents of the form $\Sigma :: \mathcal{P} \vdash G$ where $\mathcal{P}$ is a **logic program** - a set of $\mathcal{D}$ -formulas

The proof search problem for SLIM is undecidable

This is true even for **no atoms** and an **empty logic program** ⁴

⁴See Miller and Viel, *The Undecidability of Proof Search when Equality is a Logical Connective*, 2021

Our Work: Automating Proof Search

Since proof search is undecidable, we have two options

Since proof search is undecidable, we have two options

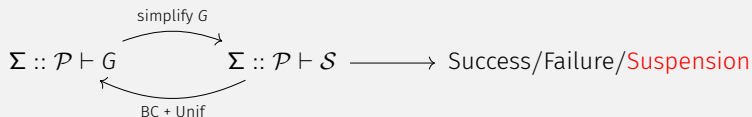
Automate as much of it as possible in the general case,
similar to **Huet's pre-unification**

Since proof search is undecidable, we have two options

Automate as much of it as possible in the general case,
similar to **Huet's pre-unification**

Identify a specific case which can be automated
completely, similar to **higher-order pattern unification**

Our Proof Search Procedure

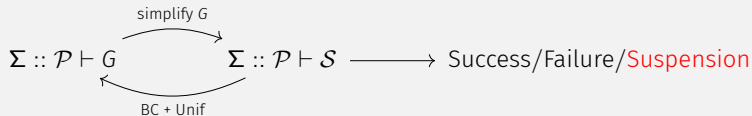


where:

BC: Backchaining steps

Unif: Unification steps

Our Proof Search Procedure



Result

All these steps preserve proofs

Simplifying goal formulas

Goal Formula Simplification

Goal Formulas $\xrightarrow{\text{normalization}}$ State Formulas $\xrightarrow{\text{reduction}}$ Reduced State Formulas

Simplifying goal formulas

State Formulas and Normalization

State formulas are a normal form for goal formulas:

$$\mathcal{S} ::= \exists \bar{h}. (G_1 \wedge \cdots \wedge G_i \wedge \cdots \wedge G_n)$$



$$G_i ::= \forall \bar{y}. (t_1 = s_1 \supset \cdots \supset t_n = s_n \supset B)$$

They are obtained by **normalization** = **raising** + using equivalences

Simplifying Goal Formulas

Raising

Raising refers to the following transformation:

$$\forall \bar{y}. (t_1 = s_1 \supset \cdots \supset t_n = s_n \supset \exists x. G)$$

$\rightsquigarrow$

$$\exists h. \forall \bar{y}. (t_1 = s_1 \supset \cdots \supset t_n = s_n \supset G[(h \bar{y})/x])$$

Simplification of Goal Formulas

Reduction and Reduced State Formulas

Select a guarded goal in a state formula $\mathcal{S}$:

$$\mathcal{S} = \exists \bar{h}. (G_1 \wedge \cdots \wedge G_i \wedge \cdots \wedge G_n)$$

Select a guarding equality in G_i :

$$G_i = \forall \bar{y}. (t_1 = s_1 \supset \cdots \supset t_j = s_j \supset \cdots \supset t_m = s_m \supset B)$$

Reduction simplifies these equalities as much as possible.

When no equality in a state formula can be simplified further, the state formula is said to be **reduced**

Simplification of Goal Formulas

Reduction Steps: Rigid-Rigid (1)

If $t_j = s_j$ occurs as

$$\dots \supset (f \textcolor{blue}{t}_{j_1} \dots \textcolor{blue}{t}_{j_k} = f \textcolor{red}{s}_{j_1} \dots \textcolor{red}{s}_{j_k}) \supset \dots$$

Replace with

$$\dots \supset \textcolor{blue}{t}_{j_1} = \textcolor{red}{s}_{j_1} \supset \dots \supset \textcolor{blue}{t}_{j_k} = \textcolor{red}{s}_{j_k} \supset \dots$$

Simplification of Goal Formulas

Reduction Steps: Rigid-Rigid (2)

If $t_j = s_j$ occurs as

$$\dots \supset (h \textcolor{blue}{t}_{j_1} \dots \textcolor{blue}{t}_{j_k} = g \textcolor{red}{s}_{j_1} \dots \textcolor{red}{s}_{j_k}) \supset \dots$$

where h and g are two distinct rigid heads, then replace with

$$\dots \supset \mathbf{f} \supset \dots$$

and therefore, **succeed**

Simplification of Goal Formulas

Reduction Steps: Flex-Rigid (1)

If $t_j = s_j$ occurs as

$$\forall y_1 \dots y_j \dots y_n. (\dots \supset y_j = t \supset \dots)$$

with y_j not occurring in t , remove y_j from the prefix and substitute t for y_j in G_i

Simplification of Goal Formulas

Reduction Steps: Flex-Rigid (2)

If $t_j = s_j$ occurs as

$$\forall y_1 \dots y_j \dots y_n. (\dots \supset y_j = t \supset \dots)$$

with y_j failing the **occurs check** in t , **succeed** (since we replace $y_j = t$ with **f**, making the entire goal true)

Reasoning about Targets

When the target is an **atomic formula**

Backchaining

Backchaining replaces the target $p\ t_1 \dots t_n$ with

$$\exists \bar{u}. ((t_1 = s_1) \wedge \dots \wedge (t_n = s_n) \wedge G)$$

by choosing a definite clause $\forall \bar{u}. (G \supset p\ s_1 \dots s_n)$ with the same predicate symbol from the program $\mathcal{P}$

Reasoning about Targets

When the target is an **equality**

Reasoning about Targets: Unification

When the target is of the form $t = s$, we perform a variant of **Huet's pre-unification**

The interesting case is when unification fails: we replace **only the target** with f^5

⁵The entire goal may not fail, consider, for example,
 $\exists x.(a = x \supset (f\ a) = (g\ b))$

Conclusion

Summary

Goal: Automate proof search for a first-order logic with equality as a connective

Methodology: Consider a sufficiently expressive subset of the logic, and perform as much proof search as possible, and **suspend** when required

Contributions

A logical framework **SLIM** which has "built-in" equality

A **non-deterministic interpreter** for a logic programming engine built on top of SLIM

A novel treatment of **unification on the left of a sequent** in an intuitionistic setting⁶

⁶In a classical setting, this is equivalent to *disunification* and has been studied before

An **implementation** of an interpreter for SLIM, either as a subsystem of Abella or as a standalone program

An **implementation** of an interpreter for SLIM, either as a subsystem of Abella or as a standalone program

Extension of the SLIM fragment to include additional proof-theoretic features, for example, the ∇ -quantifier

An **implementation** of an interpreter for SLIM, either as a subsystem of Abella or as a standalone program

Extension of the SLIM fragment to include additional proof-theoretic features, for example, the ∇ -quantifier

Research into the parallels between the current work and **CLP**: the guarding equalities are similar to *constraints*

An **implementation** of an interpreter for SLIM, either as a subsystem of Abella or as a standalone program

Extension of the SLIM fragment to include additional proof-theoretic features, for example, the ∇ -quantifier

Research into the parallels between the current work and **CLP**: the guarding equalities are similar to *constraints*

Research into **specific sub-cases** which might allow further automation